Práctica 2. El Convertidor ADC y la comunicación Serie

Objetivos

El objetivo de esta práctica es diseñar un pequeño sistema para comunicarnos con el microcontrolador ESP8266

- Montar en una placa de prototipos un microcontrolador al que conectaremos un potenciómetro para realizar un divisor de tensión
- Escribir un pequeño programa que nos permitirá detectar la tensión en el cursor del potenciómetro y presentar su valor a través del monitor serie
- Testar el funcionamiento del sistema implementado y hacer las modificaciones oportunas para presentar la lectura en mV

Material

Todo el material se proporciona en la sesión de prácticas. Cada grupo de prácticas tendrá:

- Un microcontrolador ESP8266
- Un potenciómetro
- Dos resistencias
- Cables Dupont macho-macho

Introducción

Antes de comenzar montar el circuito debemos conocer el patillaje del Microcontrolador ESP8266. Para ello se proporciona, en el anexo 1, la hoja de características de este microcontrolador. Debéis tener siempre presente esta hoja ya que es la que os indica a qué corresponde cada uno de los pines del microcontrolador.

El potenciómetro es una resistencia variable. Consta de tres patillas (en el dibujo A, B y C) de las cuales la central es la que va a moverse cuando giremos el cursor.

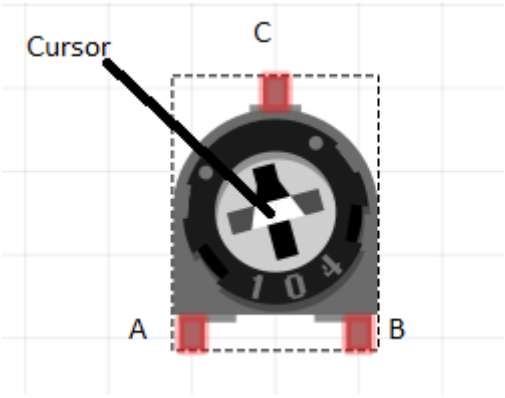


Figura 1. Potenciómetro de 10kΩ

El potenciómetro mostrado en la Figura 1 posee un valor de 10kΩ, esto significa que, entre los terminales A y B siempre hay una resistencia R_{AB} de 10kΩ, mientras que las resistencias R_{AC} y R_{CB} irán variando al mover el cursor. La suma de ambos valores (R_{AC} + R_{CB}) será igual a 10kΩ.

El convertidor analógico digital (ADC) es un circuito que recibe una señal de tensión analógica y la convierte en una señal digital. A través del siguiente video, podéis ver como es el proceso de conversión de una señal analógica a digital. En la Figura 2, se muestra un ejemplo de conversión de una señal analógica a una digital usando tres bits para codificar.

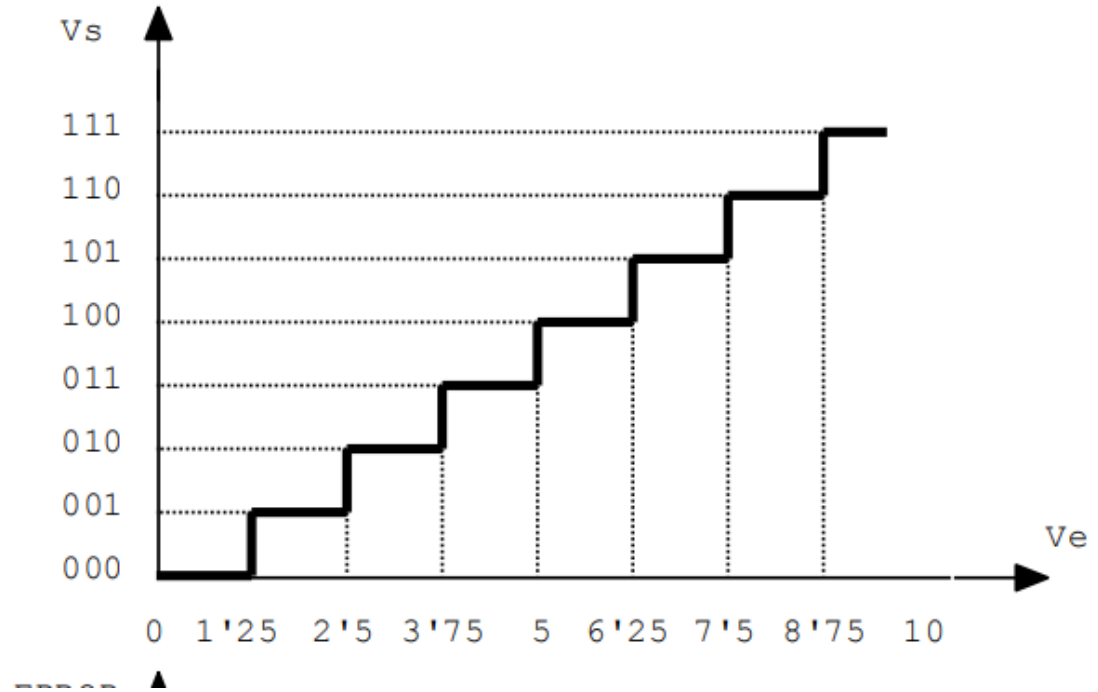


Figura 2. Conversión de una señal analógica (Ve) a una señal digital (Vs) usando tres bits de codificación

En el caso del microcontrolador ESP8266, el convertidor ADC que lleva incorporado trabaja con 10 bits, esto significa que genera códigos digitales entre 0 y 2^{10}-1 valores (0-1023). En la Figura 2 se muestra la entrada al convertidor analógico/digital del ESP8266.

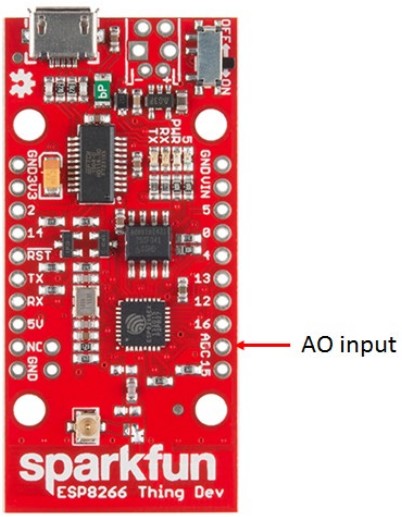


Figura 3. Pin de acceso al ADC

Las funciones que utilizaremos para trabajar con esta entrada las podéis encontrar en la página de referencia de Arduino, en el apartado Analog I/O la más importante será analogRead() la cual la utilizaremos para leer el valor de la señal analógica conectada al pin ADC.

Características del ADC incluido en el microcontrolador ESP8266

- La resolución del ADC será 1/2^10 = 1/1024 = 0.000976 V o sea 976 μV.
- El rango de medida será [0, 1[V, o lo que es lo mismo [0,1000[mV.
- Si la lectura es 0 significa que tenemos 0V en el pin ADC.
- Si la lectura nos da 1023 significa que tenemos aplicado 1023*976 μV = 999 mV en el pin ADC.

La función de Arduino map() permite mapear la lectura del ADC en el rango que nos interese. Revisad cómo funciona esta función porque la utilizaremos en nuestro programa.

Apartado 1. Implementación del Circuito

En primer lugar, montaremos el circuito de la figura en la placa de prototipos.

Las resistencias son de 10kΩ (marrón, negro, naranja) y 1kΩ (marrón, negro, rojo). El potenciómetro de 4k7.

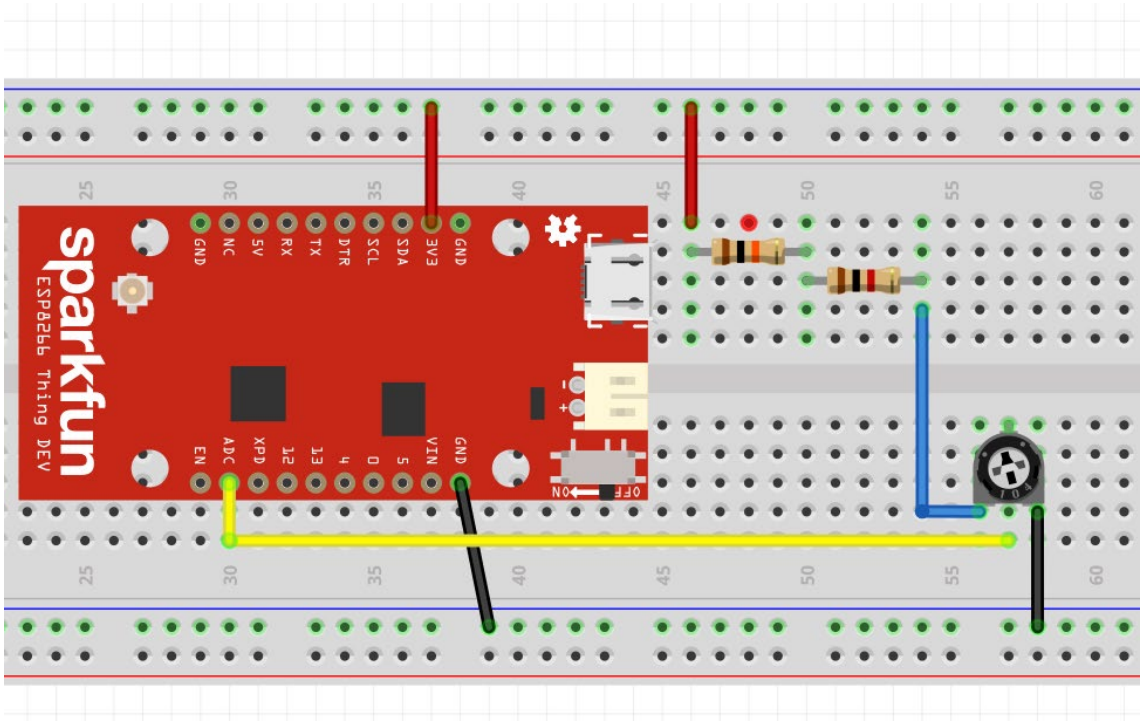


Figura 4. Circuito en la placa de prototipos

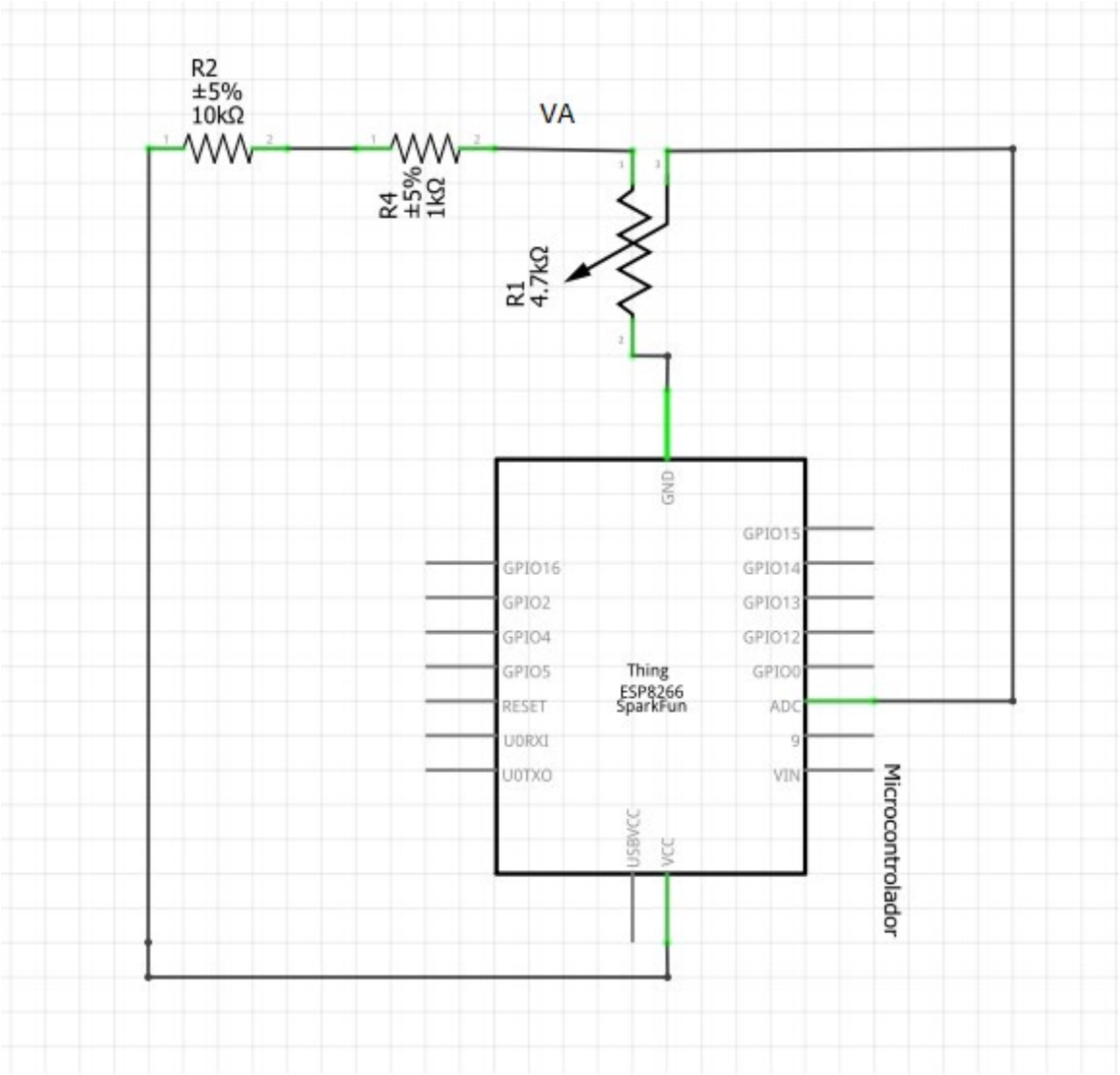


Figura 5 Esquemático del circuito de la Figura 1

Analizad el circuito de la Figura 5 para obtener qué nivel de tensión tendremos entre el punto VA y GND (V_{A0}), sabiendo que VCC son 3.3 V.

V_{A0}=

¿Es ese nivel de tensión adecuado para trabajar con el convertidor analógico digital del ESP8266? Justifique la respuesta.

Apartado 2. Escribir el código en el IDE de Arduino

Abrimos el IDE de Arduino y escribimos el siguiente programa:

ADC §

```
/*************************************************************
 * Practica 2: ADC y Comunicacion serie
 *************************************************************/

int sensorPin = A0;    // pin por el que mediremos la tension
int sensorValue = 0;  // variable para almacenar el valor leido

/* Configuracion del sistema*/
void setup() {
  Serial.begin(9600); // inicializamos la com. serie (UART) a 9600 baudios
}

void loop() {
  // leemos el valor del ADC
  sensorValue = analogRead(sensorPin);

  // Imprimimos el valor leido
  Serial.println();
  Serial.print("Voltage: ");
  Serial.print(sensorValue,DEC); // DEC para que muestre la salida en decimal
  // retardo 1 seg
  delay(1000);
}
```

Figura 6 IDE de Arduino con el programa de lectura del ADC

Conectamos el microcontrolador ESP8266 al USB y seleccionamos el dispositivo a través del menú Herramientas / Placa: **"Sparkfun ESP8266 Thing Dev".**

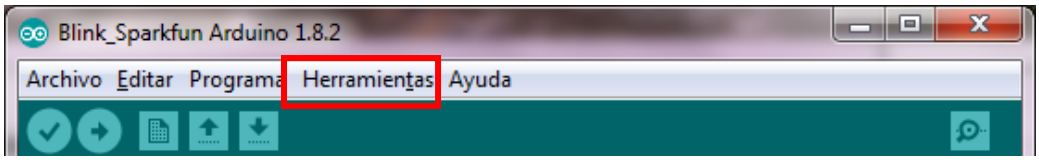


Figura 7 Menú del IDE de Arduino

Revisad que aparece este dispositivo en la parte inferior de la ventana.

El siguiente paso consiste en compilar el programa. El compilador realizará una revisión del programa que habéis escrito y os avisará si contiene algún error de programación o de otro tipo. Los errores aparecen en la ventana inferior del IDE (la zona negra).

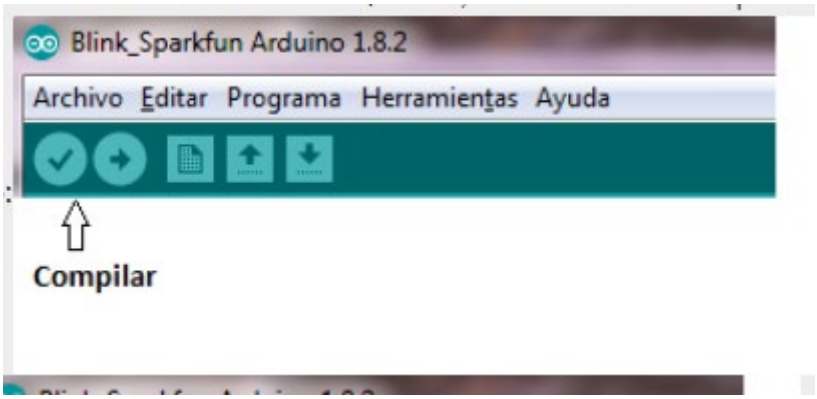


Figura 8. Menú de Compilación

Por último, una vez hayáis corregido los errores, tenéis que cargar el programa en el microcontrolador.

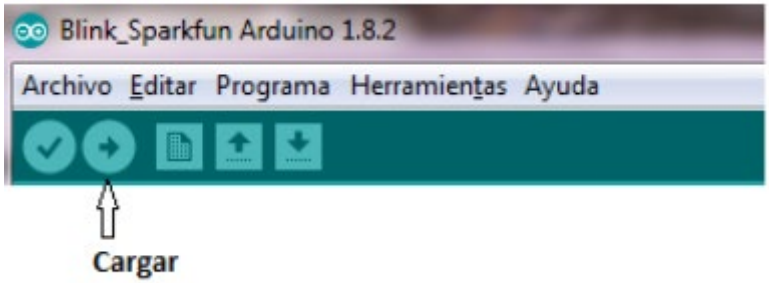


Figura 9. Menú de Carga

Apartado 3. Testeo del sistema

Para testear el sistema tendremos que abrir el monitor serie. Para ello, pulsaremos el icono que se encuentra en la esquina superior derecha del IDE (ver Figura 10).

Al abrir el monitor serie tenemos que comprobar que la velocidad de comunicación sea la que hemos programado en la configuración de nuestro programa (9600 baudios). Este ajuste lo podéis hacer utilizando el menú inferior del monitor serie (ver Figura 11).

Si hemos hecho bien todos los pasos anteriores, el monitor serie debería marcar los valores digitales que se obtienen a la salida del convertidor ADC (ver Figura 12).

Estos valores son los valores de tensión codificada en digital con 10 bits. Para obtener los correspondientes valores analógicos tendremos que utilizar la función map().

1. Modificad el programa para conseguir una lectura en mV.
2. Comprobad que la lectura obtenida coincide con la medida hecha utilizando un multímetro tal y como se muestra en la Figura 13.
3. Hallad la expresión matemática que usaríais en el caso de no poder utilizar la función map.
4. Avisad a la profesora para mostrar el funcionamiento del circuito.

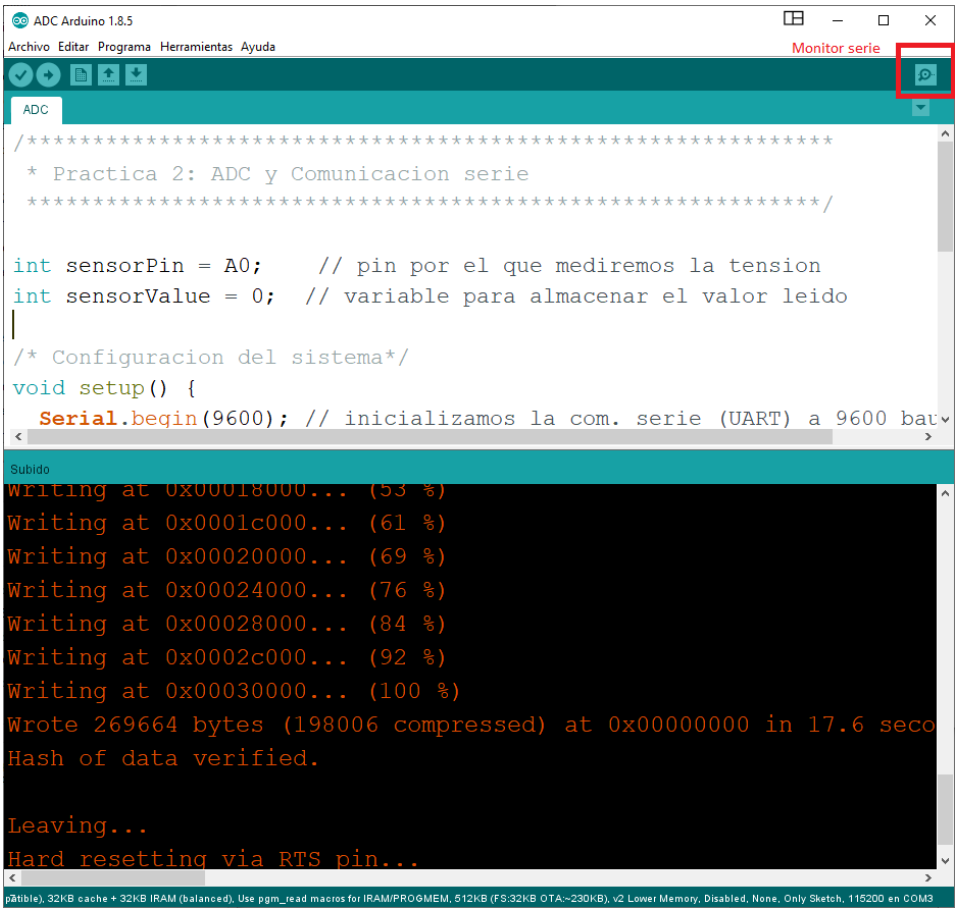


Figura 10. IDE de Arduino con el icono de apertura del monitor serie

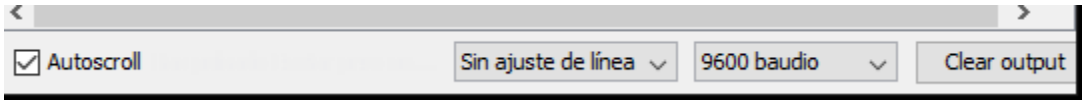


Figura 11. Menú inferior del IDE de Arduino. Ajuste de la velocidad a 9600 baudio

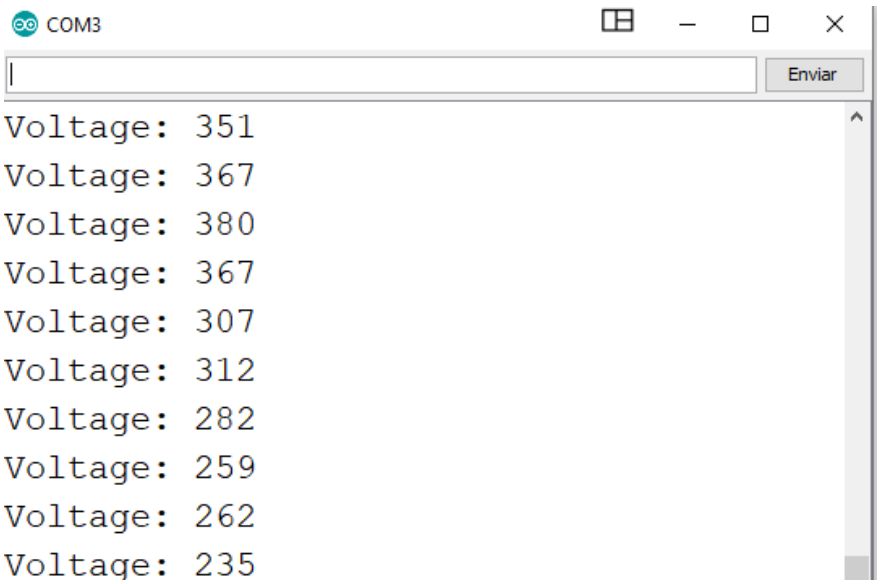


Figura 12. Lectura de la tensión digital que proporciona el ADC

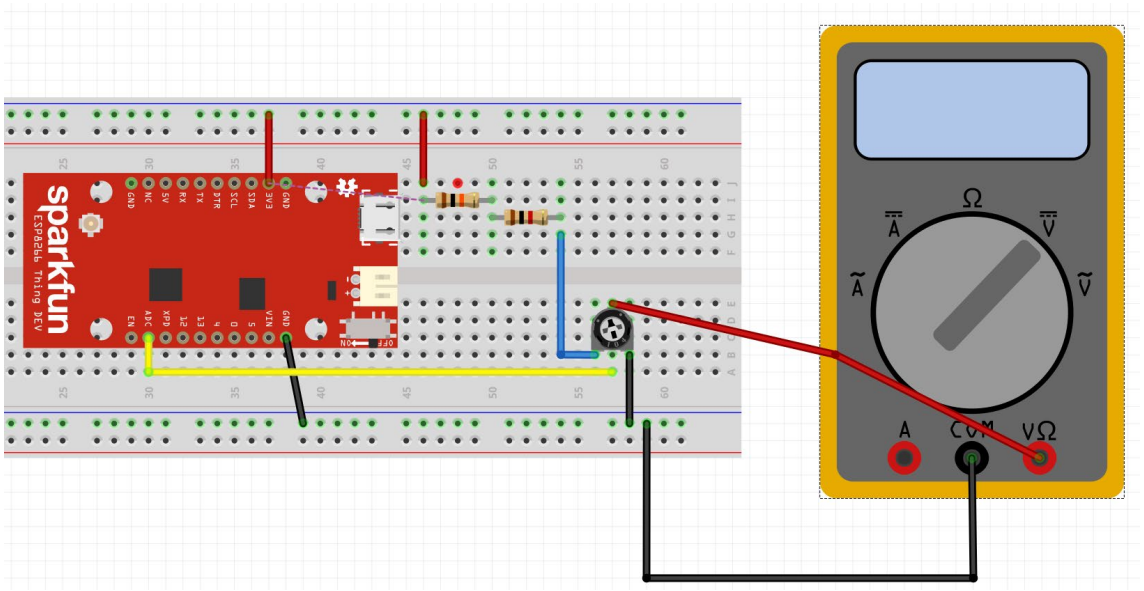


Figura 13. Medida de tensión con el multímetro para realizar el testeo

Apartado 4. Modificando el sistema

Con el siguiente apartado queremos que juguéis un poco con el programa anterior y propongáis cambios al circuito implementado. Se premiará a los equipos que consigan las implementaciones más originales.

Algunas ideas:

- Conectad un led y modificad el programa para conseguir que el led se ilumine cuando el nivel de tensión detectado supere un umbral establecido (por ejemplo, 500mV).
- Conectar varios leds de diferentes colores de manera que se vayan iluminando a medida que la tensión va aumentando. A este circuito se le llama vúmetro porque indica el nivel de voltaje de forma visual (ver Figura 14).

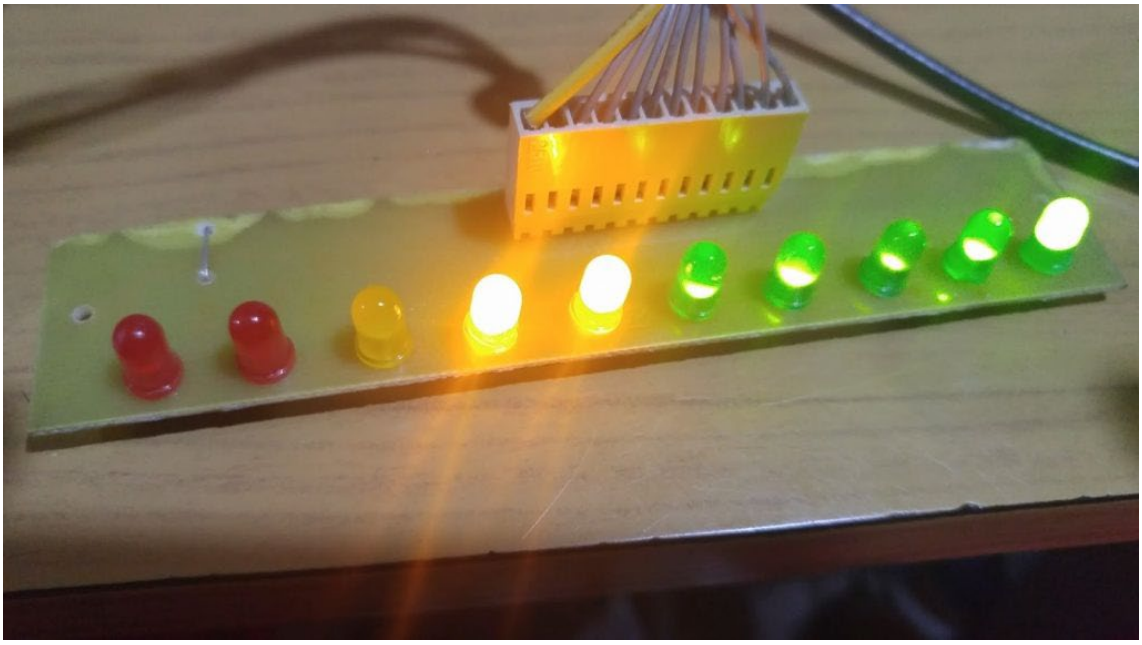
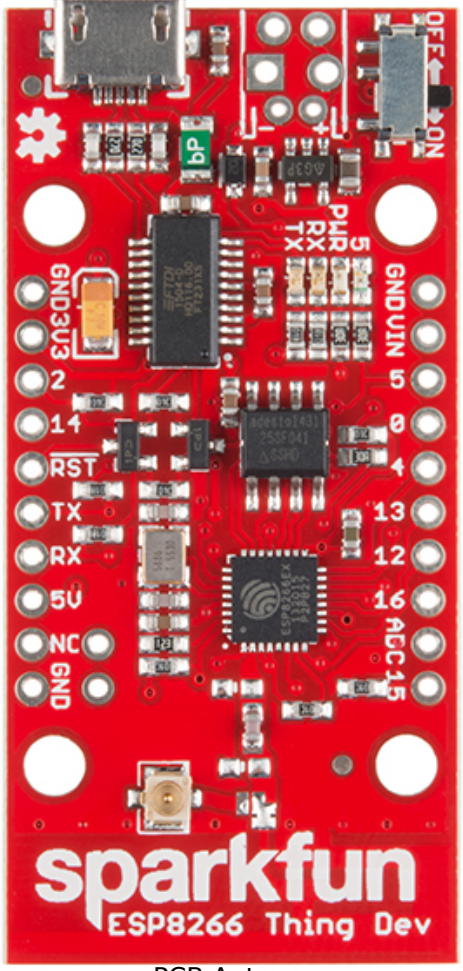


Figura 14. Vúmetro

Anexo 1: ESP8266 Thing Dev Board (WRL-13711)

Arduino add-on available
80MHz

Name	Arduino
Power	ADC
GND	Serial
Control	Misc



PCB Antenna

GPIO0
On bootup will run program if high and bootloader if low
Tied to DTR to run bootloader when reset

	GND	GND	GND	GND	
	3.3V	3V3	Vin	Vin	
SDA	D2	2	5	D5	LED
SCL/SCLK	D14	14	0	D0	Used in reset
	Reset	RST	4	D4	
TX	D7	TX	13	D13	MOSI
RX	D8	RX	12	D12	MISO
	V_USB	5V	16	D16	XPD
	Not Connected	NC	ADC	A0	10-bit 1V
	GND	GND	15	D15	CS

u.fl antenna connector
Not Connected
To use swap the solder jumper

Jumpers on back
PWR-LED - cut to disconnect power LED
SLEEP-EN - Connect RST to XPD to enable sleep (disables programming)

Power (ESP8266 Thing)
Vin: 3.7V-6V
VUSB: 5V
VCC:3.3V @500mA
Max 12mA per I/O pin

Typical Power (ESP8266 module)
Transmit 135-215mA
Receive 60-62mA
Standby 0.9mA
Deep sleep 10uA

LEDs
Power: Red
User (pin 5): Blue
RX: Yellow
TX: Green

Wi-Fi
802.11 b/g/n
Wi-Fi Direct (P2P) soft AP

sparkfun.com

CC BY SA

Hoja de Resultados Práctica 2

Al finalizar esta práctica se deberá entregar esta hoja de resultados a la profesora.

Apellidos y Nombre de todos los alumnos/as que han realizado esta práctica:

Fecha:

Apartado 1.

Analizad el circuito de la Figura 5 para obtener qué nivel de tensión tendremos entre el punto VA y GND (V_{A0}), sabiendo que VCC son 3.3 V.

V_{A0}=

¿Es ese nivel de tensión adecuado para trabajar con el convertidor analógico digital del ESP8266? Justifique la respuesta.

Una vez tengas el circuito implementado, avisa a la profesora para que lo revise y chequee este apartado.

Implementación del circuito. A rellenar por la profesora

1	2	3	4	5

Comentarios profesora:

Apartado 2 y 3

Al terminar el programa avisa a la profesora para que lo revise y chequee este apartado

Programación del circuito. A rellenar por la profesora

1	2	3	4	5

Incluir aquí la expresión matemática que sustituiría a map()

Apartado 4

Al terminar este apartado avisa a la profesora para que lo revise y chequee este apartado

Programación del circuito. A rellenar por la profesora

1	2	3	4	5

Comentarios profesora:

Comentarios alumnos: incluid aquí los problemas encontrados